

CreditTech: Alternative Credit Scoring for Financial Inclusion

AI-Driven Lending Models for Farmers, SHGs, FPOs, and Rural Enterprises

One-line project brief: CreditTech is an AI-driven alternative credit-scoring system that turns non-traditional data — SHG repayment history, Account Aggregator bank transactions, satellite-derived crop health, bureau records, and digital activity — into explainable credit-risk scores for financially excluded farmers, SHGs, FPOs, and rural micro-enterprises, validated through a 4–5 village pilot.

This document is a project-planning artifact for the mid-term conceptual design review. It defines the problem, data strategy, ML approach, architecture, and pilot design for CreditTech, and maps directly onto the mid-term submission checklist. It is written to be convertible into a 15–20 slide presentation and to serve as the working specification the team implements against post mid-term.

Document type	Ideation & Conceptual Design Plan (planning artifact — not an implementation report)
Status convention	Green text = planned/proposed. Unmarked text describing code, files, or modules refers to work already scaffolded in the team repository. No claim of production-readiness is made anywhere in this document.
Companion artifacts	Credittechplanningcontext.md (full engineering roadmap), docs/phase0/* (regulatory specs), docs/research/external_benchmark_analysis.md (comparative review of related open-source projects)

0. Mid-Term Checklist Coverage Map

Every item on the mid-term submission checklist is addressed in this document. This table is a direct index — useful both for the professor's review and as the outline for the 15–20 slide deck.

Mid-term checklist item	Covered in
Problem & village-level need assessment	§1.1–1.2
Stakeholder & institutional mapping	§1.3
Existing system (As-Is)	§1.4
Proposed system (To-Be)	§1.5
Complete system architecture	§11
Data architecture & data sources	§2
Technology stack	§12
API/DPI & institutional integration framework	§13
Functional modules	§11.2
User journey / process flow	§7.2
Security, privacy & regulatory framework	§8.5, §14 (deployment security notes)
Initial 4–5 village pilot design	§15
Preliminary cost & resource requirements	§16
Literature, policy & real-world case evidence	§17

Additional sections beyond the checklist (EDA plan, preprocessing, feature engineering, ML strategy, credit-score design, evaluation, testing plan, roadmap, folder structure) are included per the team's own technical planning requirements and support items 1–4 (start implementation, demonstrate data/ML/testing/deployment maturity).

1. Problem & Objective

1.1 The rural credit-access problem

An estimated large share of India's rural households — farmers, Self-Help Group (SHG) members, Farmer Producer Organisation (FPO) members, and micro-enterprises — are **credit-invisible or thin-file**: they have little or no formal bureau history, irregular/seasonal cash flow tied to crop cycles, and assets (land, livestock) that are hard for a conventional bank process to verify quickly. The result is not "high risk" so much as **unscored** — lenders default to rejection or manual, slow, high-cost field underwriting rather than absorbing the ambiguity.

1.2 Why traditional credit scoring fails here

Traditional scoring assumption	Why it breaks for this population
Stable monthly income	Farm income is seasonal (Kharif/Rabi/Zaid cycles), not monthly-salaried
Formal bureau history (CIBIL etc.)	Most borrowers are first-time formal borrowers — no trade lines to score
Collateral-based lending	Land titles are frequently informal, leased, or community-held; formal collateral registration is slow and often absent
Branch-based, in-person underwriting	Physical field visits are slow (days–weeks) and don't scale to thousands of small-ticket loans
Individual credit history	Ignores strong <i>group-level</i> repayment discipline signals that already exist in SHG/FPO ledgers

1.3 Stakeholder & institutional mapping

Stakeholder	Role	Interaction with CreditTech
Farmer / SHG member / micro-entrepreneur	Borrower	Gives consent, provides data, receives score + reason codes, can appeal
Bank Sakhi / Business Correspondent	Assisted-channel field operator	Submits applications on borrower's behalf via offline-first mobile UI
SHG / FPO federation	Group-level data source	Provides repayment ledgers, meeting attendance, savings records
Partner Regulated Entity (bank / NBFC / MFI)	The actual lender	Consumes the score + reason codes via a handoff API; makes the final lending decision — CreditTech never disburses funds itself
Loan officer (at partner RE)	Internal system user	Reviews score, evidence, and reason codes; approves/rejects/requests more info; every override is logged
Licensed NBFC-Account Aggregator	Regulated data-sharing intermediary	Brokers consented access to bank-transaction data under the RBI AA framework — CreditTech never touches raw bank credentials
Geospatial / bureau data vendors	External data providers	Supply satellite/weather and bureau data via contracted APIs
NABARD	Pilot sponsor / rural-finance apex body	Pilot endorsement; SHG-BLP data reference
RBI / Data Protection Board	Regulator	AA Master Directions, Fair Practices Code, DPDP Act 2023 govern consent, data use, and grievance handling
Village / Gram Panchayat administration	Local governance	Site coordination for the pilot

1.4 Existing system (As-Is)

Today, underwriting for this population is typically one of: (a) **no formal credit** — informal moneylenders at high interest; (b) **manual branch underwriting** using bureau score where it exists, biased toward rejecting thin-file applicants; or (c) **MFI/SHG-linkage lending** (SHG-BLP) that works but is slow to individualise risk and scale beyond group-liability lending. In all three, decisions take days to weeks, rely on a field visit, and produce no structured, explainable reason a rejected applicant can act on.

1.5 Proposed system (To-Be)

CreditTech is a **decision-support** system (not a lender) that aggregates consented alternative data across five rails — SHG/FPO records, Account Aggregator bank data, satellite/weather (geospatial), bureau data where it exists, and manual field entry — into a "5 Cs" feature set (Character, Capacity, Capital, Collateral, Conditions), produces a calibrated risk score with human-readable, bilingual reason codes, and hands a structured recommendation to a human loan officer at the partner lender. The system never holds credit risk or disburses funds; it shortens and de-biases the underwriting decision, it does not replace it.

1.6 Target users & expected output

Target users: farmers, SHG members, FPO members, and rural micro-enterprise owners (borrowers, assisted by Bank Sakhis where needed); loan officers at the partner RE; NABARD/RE management (portfolio oversight); the CreditTech engineering/ML team.

Expected output per application: a calibrated probability of repayment, a 0–100 and 300–900 bureau-equivalent score, a Low/Medium/High risk category, the top-5 human-readable reason codes in the local language, and a structured handoff record the partner RE's system can consume — *not* an automated approve/reject decision.

2. Data Identification & Data Sources

Category	What we need	Realistic source	Type
Financial data	Bank balance, credit inflows, account age	Account Aggregator (Sahamati ecosystem — e.g. Setu/Finvu/CAMS AA sandboxes for dev)	Real (sandbox)
Transaction / repayment	UPI frequency/volume, existing loan repayment status	AA transaction pull; partner RE's historical bureau-scored portfolio, if shared	Real / Public
Agricultural / farm data	NDVI, crop health, rainfall deviation, irrigation access	Sentinel-2 via Copernicus/Google Earth Engine (free); commercial vendors (e.g. SatSure, CropIn) for production SLA	Public / Real (vendor)
Business data	GST filings, business registration status	GSTN (for GST-registered FPOs/enterprises); Udyam Registration portal (MSME)	Public
SHG / FPO data	Repayment rate, meeting attendance, savings consistency, NABARD grade	NABARD SHG-Bank Linkage Programme (SHG-BLP) statistics (aggregate, public); individual SHG passbooks digitised by Bank Sakhi (field, semi-manual)	Public (aggregate) / Real (field)
Alternative / non-traditional data	Mobile recharge pattern, UPI regularity, utility bill timeliness	Via AA connector (financial-data scope only — no call/SMS metadata, by design)	Real (sandbox)
Government / public datasets	PM-KISAN status, PMFBY enrollment, KCC holding	PM-KISAN & PMFBY beneficiary portals, data.gov.in, NAFIS (NABARD All-India Rural Financial Inclusion Survey)	Public

Weather / crop / market data	Rainfall deviation, mandi price, crop production stats	IMD gridded rainfall (public), Agmarknet / e-NAM (public), Bhuvan (ISRO) crop maps	Public
Demographic / geographic	Village/district, agro-climatic zone, landholding band	Census 2011 village-level data, state e-Gram Swaraj / SDG portals	Public

2.1 Synthetic / proxy data — where real data isn't available yet

Pre-pilot, CreditTech has **no real repayment outcomes** — this is expected and already reflected in the design (§6.3). Two proxy strategies are used, both already scaffolded in the repository:

- **Internal mock API server** (`services/core/ingestion/mocks/server.py`) — generates synthetic AA, geospatial, and bureau payloads so the ingestion pipeline can be built and demoed end-to-end before real vendor credentials exist.
- **Pilot simulation script** (`simulate_pilot.py`) — synthesises a cohort of borrowers to exercise the full consent → ingestion → score → decision flow for demos and load testing.
- **Public benchmark datasets for modeling-technique validation only** (not for the actual pilot score): Kaggle *Home Credit Default Risk* (closest analog — alternative-data-style thin-file scoring), UCI/Kaggle *German Credit Data*, Kaggle *Give Me Some Credit*. These validate the modeling pipeline (encoding, evaluation, fairness tooling) on labelled data; they are **not agricultural** and must never be presented as CreditTech's own results.

2.2 Data limitations & handling

Limitation	Handling strategy
No real repayment labels until post-pilot (§9)	V1 model is a deterministic, literature-calibrated scorecard, not a trained classifier; real training begins once ≥200 resolved pilot outcomes exist
Any single rail may be unreachable (rural connectivity, vendor downtime)	Async orchestrator with independent per-rail timeouts and graceful degradation — a missing rail lowers confidence, it doesn't block scoring
SHG/FPO records are paper-based at many sites	Semi-manual Bank Sakhi entry UI with dual-entry reconciliation, not assumed to be API-available at launch
Geospatial vendor contract not yet signed	Open-data fallback (Sentinel-2/Earth Engine) validates the connector and NDVI features in parallel with vendor negotiation
Explicit fairness risk in raw data	Caste, religion, health, and ethnicity are never collected as features; geography is monitored for proxy-discrimination, never used as a raw model input

3. EDA Plan

The EDA plan below is designed to run against the 5-Cs feature table (§5) once pilot or proxy data is available. It is a **planned analysis pipeline**, not yet-executed results.

EDA step	What it checks
Data quality checks	Schema conformance, type validation, allowed-range checks per feature (mirrors the Pydantic schemas already used at ingestion)
Missing-value analysis	Missingness rate per feature per rail; distinguish "rail unreachable" (structural) from "field genuinely unknown" (informative)
Outlier detection	IQR / z-score flags on continuous features (e.g. <code>bank_balance_avg_6m</code> , <code>estimated_crop_income_kharif</code>); domain-sanity bounds (e.g. land holding > 50 acres flagged for review, not auto-dropped)
Duplicate detection	Duplicate borrower applications by Aadhaar-hash + village; duplicate SHG entries from dual-entry reconciliation
Distribution analysis	Histograms/KDE per feature; compare Kharif vs. Rabi income distributions
Correlation analysis	Feature–feature correlation (multicollinearity check ahead of the logistic baseline); feature–target correlation once labels exist
Class imbalance	Expected default-rate imbalance in the eventual training label; plan resampling/weighting strategy (§6) rather than assuming balance
Target-variable analysis	Distribution of the defined default proxy (DPD90+ or missed-cycle repayment, §5.9); base rate by segment
Segment-wise analysis	Separate distributions for Farmers / SHG members / FPO members / rural enterprises — these populations have structurally different feature availability and income shape
Temporal analysis	Kharif/Rabi/Zaid seasonality in income and NDVI features; month-over-month UPI activity trend
Geographic analysis	District/agro-climatic-zone level aggregates — <i>for monitoring only</i> , never fed back as a raw scoring feature (fairness constraint, §1.2/§8.5)
Feature–risk relationship	Univariate risk trend per feature (e.g. does <code>shg_repayment_rate</code> monotonically relate to the default proxy) — informs WoE binning (§5)

Required visualisations

Missingness heatmap · feature distribution histograms (by segment) · correlation matrix · NDVI/rainfall seasonal trend lines · target base-rate bar chart by segment and district · feature-vs-default-rate bin charts (WoE plots).

Expected insights

Which rails are most often missing at which sites (informs Bank Sakhi prioritisation); which features carry real signal vs. noise before committing to the candidate model; whether any feature proxies for a protected attribute (fairness check, done *before* modeling, not after).

4. Data Preprocessing Pipeline

Raw Data --> Data Validation --> Cleaning --> Missing-Value Treatment --> Outlier Handling
--> Encoding --> Scaling --> Feature Construction --> Dataset Validation --> Train/Val/Test Split

Step	Implementation	Why
Data validation	Pydantic schemas at ingestion (already in place for API payloads); range/type checks per feature	Reject malformed data before it reaches storage, not after
Cleaning	Standardise units (₹, acres), normalise categorical labels (e.g. crop names), de-duplicate	Inconsistent raw formats break both EDA and modeling silently
Missing-value treatment	Rail-aware imputation: median/mode within segment for MCAR-like gaps; explicit "missing rail" indicator flag preserved as a feature (missingness itself can be informative — e.g. no bureau history is expected for a genuinely thin-file applicant)	Naive mean-imputation would hide a structurally important signal
Outlier handling	Winsorise continuous features at domain-sane percentiles; never silently drop a borrower row	Rural income data is legitimately skewed — aggressive trimming would bias against real high-income farmers
Encoding	WoE encoding for the candidate model (§5, §6); one-hot/ordinal for the logistic baseline where WoE isn't yet applied	WoE keeps encoding regulator-legible and monotonic with risk
Scaling	Standardisation for the logistic scorecard's continuous inputs; tree-based candidates (XGBoost/LightGBM) don't require scaling	Logistic regression is scale-sensitive; boosted trees are not
Feature construction	Derived ratios (e.g. <code>kcc_utilization_pct</code> , <code>income_stability_cv</code>) computed once at this stage, versioned alongside raw inputs	See §5 for the full feature catalogue
Dataset validation	Post-pipeline schema + range re-check; row-count and null-rate assertions (this becomes the <code>tests/test_features/</code> suite, §10)	Catches pipeline bugs, not just raw-data problems
Train/val/test split	Stratified by segment (Farmer/SHG/FPO/Enterprise) and, once labels exist, by outcome; time-aware split by season to avoid leakage across a crop cycle	A random split would leak information across the same season/crop cycle

5. Feature Engineering

CreditTech's feature catalogue is already drafted (`docs/phase0/preliminary_feature_list.md`, 40+ features under a "5 Cs" framework). The table below summarises each category with the reasoning required for the mid-term review.

Category	Representative features	How calculated	Why it predicts credit risk
Character	SHG repayment rate, meeting attendance %, UPI transaction regularity	% on-time / std-dev of behaviour over trailing 12 months	Behavioural discipline is a strong repayment-intent signal even without bureau history
Capacity	Estimated Kharif/Rabi crop income, income-stability CV, PM-KISAN regularity	Geospatial yield proxy × mandi price; coefficient of variation of monthly inflows	Cash-flow stability matters more than absolute income for seasonal borrowers
Capital	SHG cumulative savings, 6-month avg/min bank balance, asset score	Ledger sums; AA balance history; weighted index of land+livestock+equipment	Savings buffer indicates shock-absorption capacity
Collateral	Land holding (acres), ownership type, NDVI-based land quality	Manual/DigiLocker entry; satellite NDVI average	Even informal land signals recoverable value and cultivation seriousness
Conditions	NDVI trend, rainfall deviation %, crop insurance enrolment	2-season satellite trend; IMD deviation from block normal	External agro-climatic shocks are the dominant risk driver for agri-linked income
Digital activity	UPI monthly count/volume, mobile recharge average, account age	AA transaction aggregation	Digital footprint proxies for formal-economy integration and stability

5.1 Fairness, privacy, leakage & regulatory flags

Concern	How it's handled
Protected-attribute leakage	Caste, religion, health, ethnicity are never collected as features, by design (not merely excluded post-hoc)
Geography as a proxy	Village/district/agro-climatic zone are computed and <i>monitored</i> for demographic parity, never used as a raw scoring input
Gender / landholding	Monitored dimensions in the fairness audit, not model features driving the score directly for demographic-parity purposes
Temporal leakage	Post-loan repayment behaviour must never enter pre-loan feature computation — enforced by the time-aware split (§4) and a dedicated leakage test (§10)
PII exposure	PII (name, phone, Aadhaar reference) is schema-separated from the feature table from day one; structured logs are scrubbed of PII automatically
Consent scope creep	Every ingestion pull is checked against an active, purpose-scoped consent record before it runs — a feature cannot be computed from data pulled outside its consented purpose

6. Machine Learning Model Strategy

Model	Role	Verdict
Logistic Regression (weighted "5 Cs" scorecard)	Baseline / V1 production fallback	Interpretable by construction; deployed first since no training labels exist pre-pilot (deterministic, literature-calibrated weights, not yet trained)
Decision Tree	Diagnostic only	Useful for quick univariate risk-splits during EDA; too unstable alone for production
Random Forest	Candidate benchmark	Good sanity-check ensemble; generally outperformed by boosting on tabular data at this scale
XGBoost / LightGBM	Primary candidate (post-pilot)	Best expected accuracy/AUC on tabular alternative data; already listed as an optional dependency for this exact purpose
CatBoost	Optional alternative	Strong native handling of categorical features (crop type, SHG grade); evaluate if XGBoost/LightGBM underperform on categoricals
Neural networks	Not justified for V1	Tabular data at pilot scale (hundreds–low thousands of rows) does not benefit from deep learning; adds opacity the regulatory context doesn't reward

6.1 Baseline vs. candidate

Baseline (deployed at pilot launch): the deterministic weighted logistic scorecard — no training data required, fully interpretable, doubles as the low-confidence fallback path even after a trained candidate is promoted.

Primary candidate (post-pilot): XGBoost/LightGBM trained on WoE-encoded features once real repayment outcomes exist (≥ 1 full repayment cycle, ≥ 200 resolved cases). The candidate is promoted only if it beats the baseline on AUC/KS *and* passes the fairness gate (§9) — beating baseline accuracy alone is not sufficient.

6.2 When to prefer the simpler, interpretable model

Whenever confidence in a prediction is low (partial-data scoring, e.g. two of four rails missing), or when a fairness-gate check on the candidate is inconclusive, the system falls back to the interpretable logistic scorecard rather than trusting a boosted-tree score it cannot yet defend to a regulator or a rejected applicant.

6.3 Problem framing

Task	Used?	Why
Binary classification (default / no default)	Yes	Core modeling task once labels exist
Probability of Default (PD)	Yes	The model output is a calibrated probability, not a hard label — needed to build the score (§7)
Credit-risk score / risk categories	Yes	PD is transformed into a 300–900 bureau-equivalent score and Low/Medium/High category (§7)
Regression	No (V1)	Loan-amount/limit regression is a plausible V2 extension, out of MVP scope
Clustering / segmentation	Diagnostic use only	Used in EDA to understand Farmer/SHG/FPO/Enterprise sub-populations, not as a production model
Anomaly detection	Diagnostic use only	Flags suspicious/inconsistent applications for manual review, not a scoring model

7. Credit Score Design

```
Applicant Data --> Alternative Data (AA/Geo/Bureau/SHG) --> Feature Engineering
--> ML Model (baseline scorecard, or candidate post-pilot) --> Probability of Default (PD)
--> Score Calibration --> Risk Score (0-100, 300-900) --> Risk Category (Low/Med/High)
--> Reason Codes --> Lending Recommendation to Loan Officer
```

PD is transformed via log-odds calibration (Points-to-Double-Odds = 20, base odds 50:1 at score 600) into a 0–100 score and a familiar 300–900 bureau-equivalent scale, then bucketed into Low / Medium / High risk categories with defined thresholds. **This is**

a **decision-support score, not an approval** — the output is a recommendation with evidence, handed to a human loan officer (§1.6, §8.6).

7.2 User journey / process flow

1. Borrower approaches Bank Sakhi or opens app -> 2. Aadhaar e-KYC + AA consent capture
3. Consent recorded (hash-chained, revocable) -> 4. Parallel data pull (AA / Geospatial / Bureau / SHG)
5. Graceful degradation if a rail is unreachable -> 6. Feature computation + scoring
7. Score + confidence band + bilingual reason codes -> 8. Loan officer reviews on decision dashboard
9. Officer approves / rejects / requests more info (override reason logged if any)
10. Structured handoff sent to partner RE -> 11. Borrower notified; appeal channel available

Not claimed: that this score is regulator-approved or production-ready. It is a pilot-stage decision-support output, subject to the fairness gate, human review, and the external regulatory sign-offs tracked separately (§17.2).

8. Interpretability & Responsible AI

Mechanism	Approach
Global explanations	Feature-importance ranking across the 5-Cs categories; published alongside each model version in the model registry
Local explanations	Per-application Shapley-style contribution values (exact for the linear baseline; the standard shap library's TreeExplainer planned for the boosted-tree candidate once it exists)
Reason codes	Top-5 SHAP-style contributions mapped to a bilingual (English/Hindi) human-readable template library, drafted per pilot-state language
Bias / fairness analysis	Weekly batch audit of demographic parity + loan-officer override rate across gender, landholding band, and geography
Sensitive-feature handling	Caste/religion/health/ethnicity never collected; geography monitored, never a raw feature (§5.1)
Data privacy	PII/feature schema separation; DPDP-aligned storage-limitation worker that anonymises PII once consent lapses
Consent	Append-only, cryptographically hash-chained consent ledger — create/verify/revoke/audit, purpose-scoped per data rail
Human-in-the-loop review	Every score reaches a loan officer with evidence and reason codes before any lending decision; overrides are logged, not silently accepted

8.6 How a loan officer understands a score

The officer's dashboard shows the score, its risk category, and the top-5 reason codes in plain local- language sentences (e.g. "SHG repayment rate is 92% — positive factor" / "NDVI trend declining over last 2 seasons — caution factor"), each traceable back to a specific feature and its contribution — not a black-box number.

9. Model Evaluation Framework

Why accuracy alone is insufficient: the default class is expected to be rare (imbalanced), so a model predicting "no default" for everyone can score high on raw accuracy while being useless — and harmful, since it hides exactly the applicants who need a nuanced decision. Evaluation must combine ranking quality, calibration, and fairness, not a single number.

Metric / check	Purpose
Precision, Recall, F1	Class-level performance on the (imbalanced) default label
ROC-AUC, PR-AUC	Ranking quality; PR-AUC specifically because of class imbalance
Confusion matrix	False-positive vs. false-negative trade-off, made explicit
Calibration curve, Brier score	Does a predicted 80% repayment probability actually mean ~80% repay? Essential since the score is presented as a probability-derived number
False-positive / false-negative analysis	A false negative (missed default) is a credit loss; a false positive (wrongly denied) is a financial-inclusion failure — both costs are named explicitly, not averaged away
Segment-wise performance	Metrics recomputed separately for Farmers / SHG / FPO / Enterprises — an aggregate-only metric can hide poor performance on one segment
Fairness evaluation	Demographic parity + equalised-odds gap across gender, landholding band, geography — a mandatory pre-promotion gate, not a post-hoc report
Threshold selection	Chosen against the business cost of false positives vs. false negatives, not fixed at the default 0.5
Cross-validation	Stratified k-fold by segment; time-aware folds by season to respect the crop-cycle structure of the data

10. Testing Plan

Test type	Covers
Unit tests	Individual functions — scorecard math, SHAP-style contribution calc, consent hash-chain verification
Data-validation tests	Schema/range checks reject malformed or out-of-bound inputs
Feature-engineering tests	Derived features compute correctly on known inputs (e.g. WoE bin edges, ratio calculations)

Model tests	Scorecard produces a probability in [0,1]; calibration output stays in [300,900]; candidate model reproducible from a fixed seed
API tests	Consent, ingestion, scoring, handoff, and monitoring endpoints — status codes, auth, payload shape
Integration tests	Ingestion orchestrator against mocked AA/geospatial/bureau servers, including partial-rail-failure paths
End-to-end tests	Full consent → ingest → score → officer decision → handoff flow on a synthetic borrower
Edge cases	Missing/incorrect applicant data, all-rails-down scoring, duplicate applications, expired consent mid-pull
Model robustness	Score stability under small, realistic input perturbations
Fairness tests	Automated check that demographic-parity gap stays within tolerance before any model is promoted
Data-leakage tests	Assert no post-loan field can influence a pre-loan feature computation
Security / privacy tests	PII never appears in logs; access-control on PII endpoints; consent required before every ingestion pull

Five of these categories are already scaffolded as pytest suites in the repository (`tests/test_consent.py`, `test_ingestion.py`, `test_scoring.py`, `test_monitoring.py`, `test_handoff.py`, `test_simulation.py`) — the plan below extends that existing structure rather than starting from zero.

```
tests/
  test_data/          # schema, range, duplicate, missingness checks
  test_features/      # feature-engineering + WoE binning correctness
  test_models/        # scorecard math, calibration bounds, candidate reproducibility
  test_api/           # consent / ingestion / scoring / handoff endpoint tests
  test_pipeline/      # preprocessing + feature-construction pipeline, end-to-end
  test_integration/   # orchestrator against mocked rails, partial-failure paths
  test_fairness/      # demographic-parity + leakage checks (new, this plan)
```

11. System Architecture

```
Data Sources (AA / Geospatial / Bureau / SHG-FPO manual)
--> Data Ingestion (async orchestrator, per-rail timeout + retry, graceful degradation)
--> Data Storage (PostgreSQL: PII schema-separated from feature tables)
--> Data Processing / EDA --> Feature Engineering --> Feature Tables (versioned)
--> ML Training Pipeline (offline, fixed cadence) --> Model Registry (version + data-snapshot tagged)
--> Prediction / Scoring Service (in-process model load)
--> Credit Scoring Engine (PD -> calibrated score -> risk category)
--> Explainability Layer (SHAP-style reason codes, bilingual)
--> Loan-Officer Dashboard / Institutional Dashboard
--> Monitoring & Feedback (fairness audit, drift, retention worker)
```

11.2 Functional modules

Module	Responsibility
Consent Service	Create / hash-chain / revoke / verify / audit consent records (DPDP-aligned)
Ingestion Orchestrator	Parallel, per-rail-timeout data pulls with graceful degradation
Scoring Service	Feature assembly + scorecard/candidate model inference + calibration
Explainability Service	SHAP-style contribution → bilingual reason-code rendering
Handoff Service	Structured export to the partner RE per the agreed API contract
Monitoring Service	Fairness audit, DPDP data-retention/anonymisation worker
Simulation Module	Synthetic pilot-cohort generation for demos and load testing

Architecture style: a modular monolith (single deployable, domain-separated internal modules) — chosen over microservices because of team size and pilot-scale traffic; documented as a formal decision with an extraction path if the system later needs to scale past a single-digit-services footprint.

12. Technology Stack

Layer	Technology	Status
API framework	Python, FastAPI	MUST-HAVE
Data / feature tooling	Pandas, NumPy	MUST-HAVE
Baseline modeling	Scikit-learn (logistic regression, encoders)	MUST-HAVE
Candidate modeling (post-pilot)	XGBoost / LightGBM; CatBoost optional	LATER
Explainability	SHAP (open-source library)	MUST-HAVE (post-pilot candidate)
Notebooks	Jupyter (EDA, feature-engineering exploration)	MUST-HAVE
Database	PostgreSQL + SQLAlchemy (async) + Alembic migrations	MUST-HAVE
Containerisation	Docker, Docker Compose (local dev with mocked connectors)	MUST-HAVE
Version control / CI	Git/GitHub, GitHub Actions (lint, type-check, test, build)	MUST-HAVE
Infra-as-code	Terraform, single-region managed containers (no Kubernetes)	MUST-HAVE

Testing	pytest, pytest-asyncio, ruff, mypy	MUST-HAVE
Data/model versioning	DVC (once real training data exists)	LATER
Experiment tracking	MLflow	OPTIONAL — only if experiment count justifies it
Dashboard UI	FastAPI + Jinja2/HTMX (internal officer tool, 20–50 users)	MUST-HAVE
Borrower app	Android, Kotlin or React Native (offline data capture)	MUST-HAVE
Streamlit/React for analytics	—	OPTIONAL — plain server-rendered charts suffice at pilot scale
Model serving infra (Triton/KServe)	—	NOT NEEDED — in-process model load at this traffic volume

13. API / DPI & Institutional Integration

```

External Data / API (AA, Geospatial vendor, Bureau/CIC, SHG federation)
--> API Layer (per-connector client: auth, timeout, exponential-backoff retry)
--> Validation (schema + consent-scope check)
--> Feature Pipeline --> ML Model --> Credit Score --> Lending Recommendation
--> Structured Handoff to Partner RE (REST/JSON, per agreed contract spec)

```

REST/JSON is used throughout because every relevant external Digital Public Infrastructure rail (Account Aggregator, and eventually OCEN/ULI-style handoff) is itself REST/JSON — matching it avoids an unnecessary translation layer.

Explicitly flagged: actual API availability, authorisation, consent scope, and regulatory sign-off for each rail (AA FIU licensing, geospatial vendor contract, bureau CIC access, RE handoff contract) must be verified and formally agreed before any production integration — these are tracked as open external items, not assumed. See §17.2.

14. Deployment Plan (Realistic MVP, Not Full Production)

Stage	Approach
Development environment	Docker Compose locally, all four rails mocked, live-reload FastAPI + Postgres
Model training	Offline, fixed cadence (not continuous) — pilot scale doesn't justify streaming retraining
Model serialisation	Versioned artifact + training-data snapshot reference in a lightweight model registry
Backend API	FastAPI monolith, containerised
Database	Single managed PostgreSQL instance, India region
Frontend / dashboard	Server-rendered officer dashboard (Jinja2/HTMX); Android borrower app
Containerisation	Docker images built and scanned in CI (pip-audit, gitleaks)
Cloud deployment	Single managed cloud, India region, managed containers (ECS/Cloud Run) — not Kubernetes, per architecture decision (team size and traffic don't justify it)
Monitoring	Weekly batch fairness/drift job; structured, PII-scrubbed logs; managed APM (bought, not built)
Model / data drift	Season-boundary and significant-drift triggers for retraining review (human-approved before promotion, not automatic)
Retraining strategy	At most one or two retrains across a 9–12 month pilot window (one Kharif–Rabi cycle) — a fully automated retraining pipeline is explicitly premature at this stage

MVP-essential: Docker Compose dev environment, CI pipeline, single-region managed deploy, weekly batch monitoring.
Deferred: autoscaling, multi-region, real-time streaming monitoring, dedicated model-serving infrastructure.

15. Initial 4–5 Village Pilot Design

Element	Plan
Participants	~750–1,250 borrowers across 4–5 sites: farmers, SHG members, FPO members, rural micro-enterprises; onboarded via Bank Sakhi-assisted channel and/or Android app
Data collected	All five rails where reachable at each site (AA, geospatial, bureau, SHG/FPO manual entry, digital-activity signals) — a per-site connectivity/device assessment precedes onboarding
Consent	Aadhaar e-KYC + purpose-scoped, revocable AA-style consent recorded in the hash-chained ledger before any data pull
System output	Score + confidence band + bilingual reason codes, delivered to the single partner RE integrated for the pilot
User interaction	Bank Sakhi submits/assists onboarding; loan officer reviews score + evidence on the decision dashboard and records approve/reject/request-more-info with mandatory override-reason logging
Pilot success metrics	Approval-rate uplift vs. the RE's existing process; score-availability rate under partial-rail data; PAR90 on disbursed loans; reason-code comprehension (borrower survey); fairness-parity gap within tolerance

Feedback loop	Resolved repayment outcomes from site 1 feed the post-pilot candidate-model training trigger (§6.1); grievance/appeal channel outcomes reviewed alongside officer override patterns
---------------	---

Site 1 launches first and must be stable for 4+ weeks (including a production-credential smoke test on every external rail) before expansion to sites 2–5 — a staged rollout, not a simultaneous 5-site launch.

16. Preliminary Cost & Resource Requirements

Indicative figures below are order-of-magnitude planning estimates for a 9–12 month, 4–5 site pilot — **not** a committed budget. They exclude credit capital itself, which is disbursed by the partner RE, never by CreditTech.

Cost driver	Notes
Core engineering team	Largest single line item at pilot stage (backend/ML engineers, one mobile developer, field-ops coordination)
Geospatial (EO) data licensing	Per-hectare/per-query vendor pricing — scales with borrower count; open-data fallback (§2.2) reduces this cost during pilot validation
Account Aggregator per-consent fees	Scales directly with number of applications processed
Cloud infrastructure & MLOps	Grows step-wise across scale stages, not linearly — protected by avoiding Kubernetes/dedicated model-serving infra at pilot scale
Field operations	Bank Sakhi/BC coordination and device procurement at each site — a real, ongoing cost that technology reduces but does not eliminate

Indicative order-of-magnitude pilot budget: engineering team cost and combined geospatial/AA integration cost dominate the total; infra and field-ops costs are secondary but non-trivial line items. Exact figures depend on vendor contracts (still open, §2.2) and are refined once those close.

16.1 Team & roles for the mid-term / build phase

Role	Responsibility
Backend / API engineer(s)	Consent, ingestion, scoring, handoff services
ML engineer	Feature engineering, baseline scorecard, candidate model (post-pilot), evaluation/fairness tooling
Mobile developer	Android borrower app (offline-first)
Field-ops coordinator	Bank Sakhi training, device/connectivity assessment, site liaison
Project/product lead	Stakeholder coordination, regulatory-item tracking (§17.2), pilot sequencing

17. Literature, Policy & Real-World Case Evidence

17.1 Policy & regulatory grounding

- **RBI Account Aggregator (AA) Master Directions** — governs consented financial-data sharing via licensed NBFC-AAs; the basis for the AA connector and consent-scope design.
- **Digital Personal Data Protection (DPDP) Act, 2023** — basis for the hash-chained consent ledger, PII/feature schema separation, and the storage-limitation (data-retention) worker.
- **RBI Fair Practices Code** — basis for the mandatory grievance/appeal channel and SLA.
- **NABARD SHG-Bank Linkage Programme (SHG-BLP)** data and grading system — source of the SHG-grade feature and a real-world precedent for group-liability alternative underwriting at scale.

17.2 Related academic / industry work

The team's "5 Cs" feature-weighting approach draws on published alternative-credit-scoring literature (Jonnalagadda & Babu, 2026) for the initial bootstrap weights, used in the absence of real repayment labels.

17.3 Real-world case evidence (comparative review of related open-source projects)

Project	Relevant technique	Adopted into this plan?
agri-access (Indonesia)	Open-weight satellite foundation model (Prithvi-EO-2.0) against free Sentinel imagery instead of a paid vendor	Yes — informs the open-data geospatial fallback (§2.2)
Krishi-Patra (NABARD hackathon)	Dynamic, ongoing crop-health monitoring tied to loan terms; blockchain for transparency	Monitoring concept adopted as a post-pilot enhancement; blockchain not adopted — the hash-chained ledger already gives tamper-evidence at lower operational cost
Credit-Risk-Probability-Model-for-Alternative-Data	WoE/IV encoding, explicit proxy-label documentation, Basel-II framing, DVC data versioning	Yes — informs §5/§6 encoding strategy and §9 evaluation discipline

Full comparative analysis: `docs/research/external_benchmark_analysis.md`.

18. Project Roadmap

Phase	Focus	Key tasks	Deliverables	Tools
1	Problem & literature review	Stakeholder mapping, policy review, related-project benchmarking	This document	Desk research

2	Data collection	Set up AA/geospatial/bureau sandboxes; assemble public datasets; build mock server for synthetic data	Raw data pull scripts, mock API server	httpx, Requests, public portals
3	EDA	Run §3 EDA plan on proxy/synthetic data	EDA notebook + insights report	Jupyter, Pandas, Matplotlib/Seaborn
4	Data processing	Implement §4 pipeline	Cleaned, validated dataset	Pandas, Pydantic
5	Feature engineering	Build 5-Cs feature table + WoE encoding	Versioned feature table	Pandas, scikit-learn
6	Baseline models	Deploy logistic scorecard	Working scorecard + calibration	scikit-learn
7	Advanced models	Train XGBoost/LightGBM on labelled/proxy data	Candidate model + comparison report	XGBoost, LightGBM
8	Explainability & fairness	Wire SHAP; run fairness audit	Reason-code templates, fairness report	SHAP, custom fairness auditor
9	Evaluation	Run §9 metric suite + cross-validation	Evaluation report	scikit-learn metrics
10	API & deployment	Wire FastAPI services, Docker Compose, CI/CD	Deployable service	FastAPI, Docker, GitHub Actions
11	Testing	Implement §10 test suites	Passing CI pipeline	pytest, ruff, mypy
12	Pilot / validation	Site-1 staged launch, production-credential smoke test	Pilot results, feedback loop	Monitoring dashboards

19. Final Project Folder Structure

```

credittech/
  data/           # raw + processed datasets (synthetic/proxy for pilot demo)
  notebooks/      # EDA, feature-engineering exploration (Jupyter)
  src/            # shared application code
  features/       # feature-engineering pipeline (5-Cs table, WoE encoding)
  models/         # baseline scorecard + candidate model training code
  api/            # FastAPI services: consent, ingestion, scoring, handoff, monitoring
  frontend/       # officer dashboard (Jinja2/HTMX), Android borrower app
  tests/          # see §10 test structure
  configs/        # environment configs, settings
  docs/           # phase0 specs, roadmap, this document, benchmark research
  deployment/     # Dockerfile, docker-compose.yml, Terraform (infra/)
  README.md       # project overview + local setup

```

This mirrors the structure already scaffolded in the team repository (`services/core/*`, `ml/*`, `apps/*`, `infra/*`, `docs/*`) — the mid-term plan builds on existing scaffolding rather than proposing a rewrite.

Closing note — assumptions & open items

This plan assumes: (1) AA/geospatial/bureau sandbox access can be obtained during the build phase even if production contracts are still pending; (2) real repayment-outcome data will not exist until after at least one pilot repayment cycle, so the V1 model is deliberately a non-trained, literature- calibrated scorecard; (3) a single partner RE integration is sufficient for the pilot. Open, externally- tracked items (signed RE Letter of Intent, AA FIU credentials, geospatial vendor contract, NABARD endorsement, legal sign-off on the consent flow, Bank Sakhi device procurement) are regulatory/business dependencies outside the team's direct control and are tracked separately, not assumed resolved.